LabLens: Reading, Flagging and Explaining Medical Lab Reports with Trend Tracking

Banoth Swapna
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology
Hyderabad, India
[email address]

Bekkam Varshitha
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology
Hyderabad, India
[email address]

A. Navadeep Reddy
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology
Hyderabad, India
[email address]

Budigam Manoj
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology
Hyderabad, India
[email address]

L. G. Reddy
[DESIGNATION], Dept. of Computer Science and Engineering
TKR College of Engineering and Technology
Hyderabad, India
[email address]

***Abstract*---**Laboratory reports reach patients as PDFs or photographs full of abbreviations, units and reference ranges that most readers cannot interpret, and nothing links one report to the next. This paper presents LabLens, a web platform that reads an uploaded report, extracts each test name, value, unit and reference range, labels every result as low, normal or high, explains abnormal results in plain language, and plots each parameter across successive reports. Text comes from the PDF text layer, with an image-preprocessing and Tesseract fallback for scans; fields are parsed with line-level regular expressions, noisy test names are matched to a 55-name dictionary with a character n-gram TF-IDF matcher, and explanations come from a curated rule base with an optional language-model rewrite. Measured on 120 generated report PDFs in eight layouts (1,680 labelled rows), row precision is 0.963, recall 0.874 and F1 0.916; range and status accuracy are both above 99 %, while unit accuracy is 87.1% and a layout with one table cell per line yields no rows. The pipeline needs 53 ms per report. No real or manually labelled report set and no scanned-image OCR run was available, so those results are stated as gaps.

***Index Terms*---**health insights, lab report analysis, named entity extraction, optical character recognition, plain-language explanation, TF-IDF matching, trend tracking.

I. INTRODUCTION

A blood or urine test returns a table of parameters, results, units and reference ranges written for clinicians. The patient who receives it as a PDF or a phone photograph usually cannot tell which values matter, so understanding the report costs a follow-up consultation, and the delay and fear that come with it. Manual reading by staff is slow and error-prone, and a single report cannot show whether a parameter is improving or worsening. Recent work therefore either digitises image-based lab reports [2], asks general vision-language models to interpret them [3], or generates patient-oriented explanations and actions with language models [1], [4].

Existing tools digitise reports but do not explain them, and the explanation systems are narrow: the base paper [1] applies retrieval-augmented generation to one condition, metabolic syndrome, and its evaluation measures the quality of generated answers rather than the correctness of the values read from the report. No history is kept, so trends cannot be seen. The gap addressed here is a general platform for arbitrary lab parameters that extracts, flags, explains and tracks.

The contributions are: (1) a complete, tested pipeline from upload to a stored, flagged and explained result, served by a FastAPI layer and a React interface; (2) a labelled evaluation harness, with a generated report corpus of 120 PDFs, that measures extraction, name matching, classification, explanation coverage and latency (36 automated tests accompany it); (3) a measured account of where the extractor fails, with the failures traced to specific lines of the code; and (4) longitudinal trend and report-comparison views over the stored results. Section II reviews related work, Section III describes the method, Section IV reports the results and Section V concludes.

The project set six objectives: (1) accept reports as PDF or scanned image and extract their text with high-speed OCR; (2) identify test parameters, values, units and reference ranges; (3) compare each value with its range and flag abnormal results; (4) explain abnormal findings in plain language grounded in trusted sources; (5) track parameters across successive reports; and (6) present the results on a dashboard and evaluate extraction accuracy against manually labelled reports. Section V reports how far each was met.

A. System Hardware

All measurements were taken on one desktop: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, no GPU, Windows 11 Pro.

B. Software Requirements

Python 3.11.9, PyMuPDF 1.28.2 [5], OpenCV 4.11.0 [6], scikit-learn 1.5.0 [7], SQLAlchemy 2.0.30, FastAPI 0.143.3 [8] on uvicorn 0.54.0, Node.js 24 with React 18.3.1, Vite 5.4.21 and Recharts 2.15.4 [9]. Tesseract [10] is used by the code for scans but is not installed on this machine, and no language-model key was set.

II. LITERATURE SURVEY

Base paper. Karhade et al. [1] build a retrieval-augmented generation (RAG) system [11] on the LLaMA-3 8B model that explains blood-test results with a focus on metabolic syndrome. The knowledge base holds 20 medical books, 383 transcribed videos and a document that defines 33

blood parameters with normal, optimal and possible ranges. Text is extracted with PyMuPDF, split into 1,024-character chunks with a 100-character overlap, embedded and stored in ChromaDB; the top three chunks above a similarity threshold are added to the prompt, sources are returned with the answer, and a generic answer is given when nothing relevant is retrieved. Twenty responses were scored from 1 to 100 by GPT-4 for correctness, clarity, helpfulness and completeness; the paper states averages of 92.4, 89.7, 91.1 and 88.9, whereas the values in its own result table average 94.0, 94.9, 90.1 and 88.7. It reports no extraction accuracy or latency and lists multilingual support, longitudinal personalisation and speech input as future work.

Digitising and interpreting reports. Ren et al. [2] argue that lab reports photographed by patients for teleconsultation should be turned into structured, stored data and propose a serialisation method for image-based reports. Bakhsheshi and Beigy [3] prompt three general vision-language models with questions about 100 blood-report images and compare the answers with Sentence-BERT [12]; they caution that the dataset is small. Checkup2Action [4] turns 2,000 de-identified check-up reports into structured action cards with priority, department and follow-up window; experts judged 84 % of evaluated outputs fully reasonable, and removing the safety constraints raised problem recall from 0.527 to 0.825 at the price of more diagnostic overstatement.

Language models for medical text. Beyond the base paper, Salvagno et al. [13] examine the feasibility of ChatGPT in clinical and research scenarios, Liu et al. [14] study AI-generated draft replies in health records, and MEDPLAN [15] uses a two-stage RAG design that first assesses and then plans treatment. Medalyze [16] fine-tunes three FLAN-T5-Large models to summarise medical reports and extract health concerns behind a web and mobile interface. All of them generate text; none checks the numbers it was given. Table I sets the closest works side by side.

TABLE I
REPRESENTATIVE WORK COMPARED

Ref.	Approach	Data	Reported result	Limitation
[1]	LLaMA-3 8B + RAG, ChromaDB	20 books, 383 videos; 20 test answers	Mean GPT-4 scores 88.7-94.9 of 100	One condition; no extraction test
[2]	Serialising image-based lab reports	Photographed reports	Not stated in the abstract read	Explanations not covered
[3]	General vision-language models	100 blood-report images	Answer similarity only	Small set; no label check
[4]	Constrained action-card workflow	2,000 check-up reports	84 % outputs fully reasonable	No value tracking
[16]	Three FLAN-T5-Large models	Medical text	Not stated in the abstract read	Text only; no trends

Positioning. Compared with these works, LabLens makes a narrower and more checkable claim. It does not ask a language model to read the report, so the numbers on screen are the numbers on the page, and the rule table makes every statement traceable to its source file; what it gives up is breadth of explanation and tolerance of unusual layouts, which the generated evaluation below quantifies.

OCR and field extraction. PP-OCR [17] is a practical, ultra-lightweight OCR system, and the open-source Tesseract engine [10] is a widely deployed alternative; conditional random fields [18] are the classical model for labelling sequences such as the name, value and unit of a test line. The work here uses neither a trained tagger nor PP-OCR: it parses the text layer with regular expressions and falls back to Tesseract. **Gap.** The survey leaves a platform that covers arbitrary lab parameters, checks the extracted values themselves, explains abnormal results and keeps the history; the abstract's PP-OCR and CRF components were not built, as Section III states.

III. METHODOLOGY

Pipeline. An upload (PDF, JPG, JPEG or PNG, at most 15 MB) is validated for extension, size and readability. For a PDF, each page's text layer is read with PyMuPDF; a page with fewer than 30 characters is rendered at 300 dpi instead. Images and such pages are converted to grayscale, denoised (non-local means, h = 10), thresholded with a Gaussian adaptive threshold (block 31, C = 15), deskewed from the minimum-area rectangle of the text pixels and read by Tesseract in page-segmentation mode 6. Fig. 1 shows the architecture and Fig. 2 the main flow.

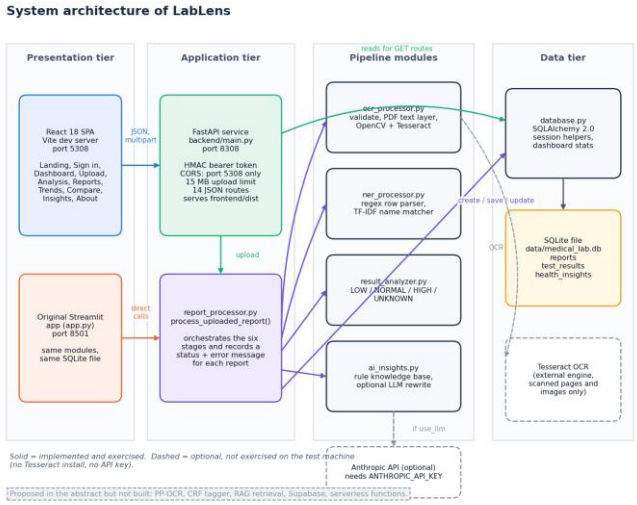


Fig. 1. System architecture: React interface, FastAPI service, shared pipeline modules and the SQLite store.

Field extraction. Lines that look like headers or patient details are skipped. A regular expression takes a test name, a numeric result, an optional unit and a reference range written as 'a - b', 'a to b', '<x' or '>x'. The date is read from the text, rows with neither unit nor range are dropped, and duplicates (same name, value and unit) are removed. A row's confidence is 0.9 with a two-sided range and 0.6 otherwise, minus 0.1 when the unit is missing.

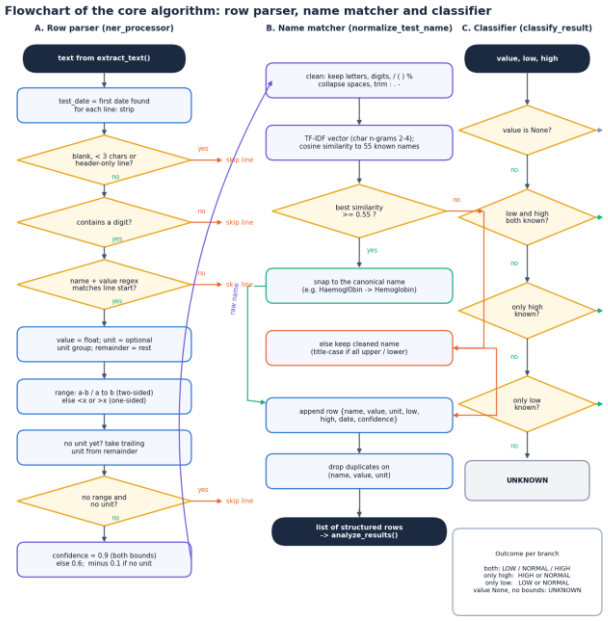


Fig. 2. Core flow of one upload: validation, text extraction, parsing, classification, insights and storage.

Name matching. Raw names are cleaned and mapped to a dictionary of 55 test names. Each name is a vector of character 2- to 4-gram TF-IDF weights [19] computed with scikit-learn [7]; a name is replaced by the dictionary entry with the largest cosine similarity, s(a, b) = a.b / (|a| |b|), only if s >= 0.55; otherwise it is kept, title-cased. The threshold is analysed in Section IV.

Classification and insights. With result v and reference bounds l and h (either may be absent), the status is LOW if l is present and v < l, HIGH if h is present and v > h, NORMAL if at least one bound exists, and UNKNOWN otherwise; bounds are inclusive. The dashboard marks a result critical when v < 0.5 l or v > 1.5 h. For each LOW or HIGH result the insight module looks the test up in a curated table of 17 tests (what it measures, general causes of a low or high value) or falls back to generic wording, and always adds a suggestion to consult a clinician and a disclaimer. An optional switch rewrites the text with a language-model API [20] (300 output tokens) and silently falls back to the rule text on any error; this switch was not exercised in the measurements. The design replaces the base paper's retrieval step [1], [11] by a fixed table: it cannot hallucinate but covers little.

Storage and interface. Reports, test results and insights are stored in three SQLite tables through SQLAlchemy with cascading deletes. The API serves the dashboard, report list and detail, trends, comparison of two reports and an insight feed; a trend's direction is decided from its first and last point. A single demo account protects the routes with a signed 12-hour token. The React interface draws trend charts with the reference range as a shaded band.

Design choices. The extractor is rule-based because no labelled training set of real reports existed, because a trained tagger would have been impossible to check against anything, and because rules run in milliseconds on a CPU and can be read line by line when a value is wrong. Character n-grams suit test names since OCR damage changes single letters, and a similarity threshold gives an explicit trade-off between repairing noisy names and renaming genuine ones. Explanations come from a fixed table because a medical statement shown to a patient should be one the team has read; the cost is coverage, and the optional rewrite is the only path by which new wording can enter.

Errors and testing. A failure in any stage marks the report as failed, stores a readable message and keeps the text read so far; unsupported extensions, empty files, files over the limit and corrupt PDFs are rejected before processing, and a page without readable rows produces a 'no recognizable lab test rows' message rather than an empty success. The test suite covers the classification boundaries, line parsing, name matching, input validation, persistence with cascading deletes, the dashboard's critical rule, login and tampered tokens, and the upload, trend, comparison and insight routes; a separate smoke test starts its own server on a copy of the code and drives the API and a browser, so the stored data is never touched.

Evaluation data. No real or manually labelled lab reports were available, so a corpus was generated (seed 42): 120 PDFs with a text layer, 15 for each of eight layouts (plain, parenthesised range, flag letter, colon, unit last, mixed range separators, one table cell per line, and names corrupted with 10 % simulated per-character OCR noise). Each report has 12 tests from a pool of 40 dictionary tests and 2 tests outside the dictionary, header decoys and a collection-date line, giving 1,680 ground-truth rows. Values fall in range, below or above in roughly 55, 25 and 20 % of cases. A predicted row matches a ground-truth row by value; name, unit, range and status are then scored separately. Labels are generated, and the status label uses the project's own rule, so status accuracy measures propagation, not clinical truth.

Metrics. With TP the predicted rows matched to a ground-truth row by value, precision is TP over the rows returned and recall is TP over the ground-truth rows; F1 is their harmonic mean. Name, unit, range and status accuracy are fractions of the matched rows, and the exact-row rate is the fraction of all ground-truth rows whose four fields are all correct, so a missed row counts against it. Name matching is also tested alone: each of the 55 dictionary names is corrupted 30 times per noise level (substitution with a look-alike character, deletion or insertion) and compared with an exact match and with the standard-library difflib similarity ratio at 0.60. Latency is the median of repeated runs on the machine above, and the API is timed on the loopback address.

IV. RESULTS

Extraction. Over all 1,680 rows the extractor returns 1,525 rows with precision 0.963, recall 0.874 and F1 0.916 (Table II). Among detected rows, range accuracy is 99.9%, status accuracy 99.9%, name accuracy 95.6% and unit accuracy 87.1%; 72.9% of all ground-truth rows are

recovered with every field correct. Name accuracy is 98.7% for dictionary tests and 77.1% for the unseen ones. On the seven layouts with one test per line recall is 99.9%; the table layout gives zero rows because every cell reaches the parser on its own line.

TABLE II
EXTRACTION ON 210 GENERATED ROWS PER LAYOUT (1,680 IN TOTAL)

Layout	Rows out	Prec.	Recall	F1	Unit acc.	Exact row
plain	217	0.968	1.000	0.984	0.938	0.929
paren	217	0.968	1.000	0.984	0.881	0.867
flag	217	0.968	1.000	0.984	0.576	0.562
colon	217	0.968	1.000	0.984	0.909	0.886
unit-last	217	0.968	1.000	0.984	0.943	0.914
mixed-sep	217	0.968	1.000	0.984	0.933	0.909
tabular	7	0.000	0.000	0.000	--	0.000
ocr-noise	216	0.963	0.991	0.977	0.918	0.767
All	1525	**0.963**	**0.874**	**0.916**	**0.871**	**0.729**

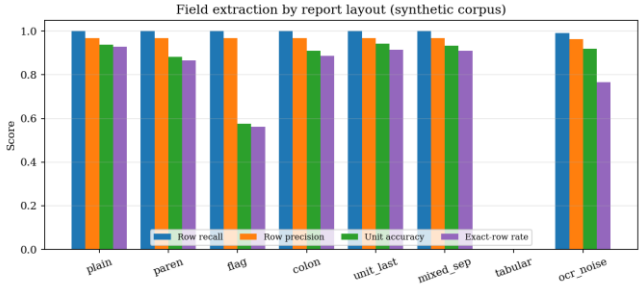


Fig. 3. Row recall, row precision, unit accuracy and exact-row rate by report layout.

Failures. The date line was misread as a test row in all 56 reports with a dd-mm-yyyy date: '12-04-2026' becomes a test called 'Sample Collected' with value 12 and range -4 to 2026. They account for 55 of the 57 false-positive rows (in one report the misread row was matched to a genuine row of the same value); the other two are stray rows. Unit errors dominate the remaining mistakes (179): the unit expression does not know mg/L, umol/L, ng/dL, uIU/mL or pg/mL, and a flag letter between value and unit (the flag layout, unit accuracy 57.6%) hides the unit. 'Sodium Bicarbonate' is renamed 'Sodium', and a test missing from the dictionary can be silently renamed. Range and status errors are rare (accuracy 99.9% and 99.9%). Figs. 4 and 5 show the screens that present the extracted rows.

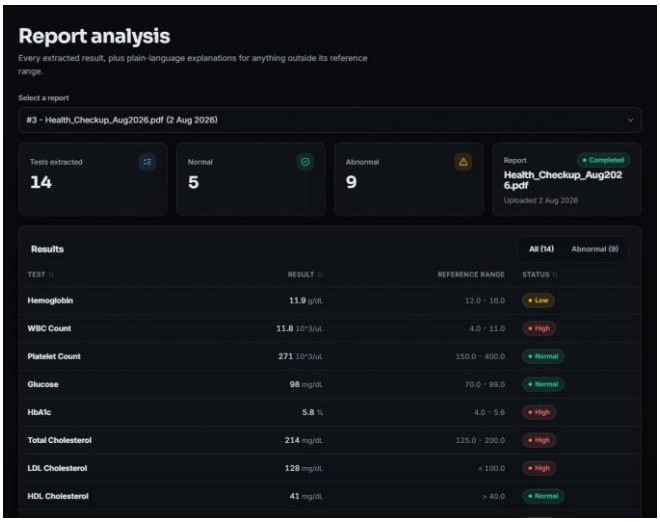


Fig. 4. Output screen: report analysis for one stored report, with the flagged results.

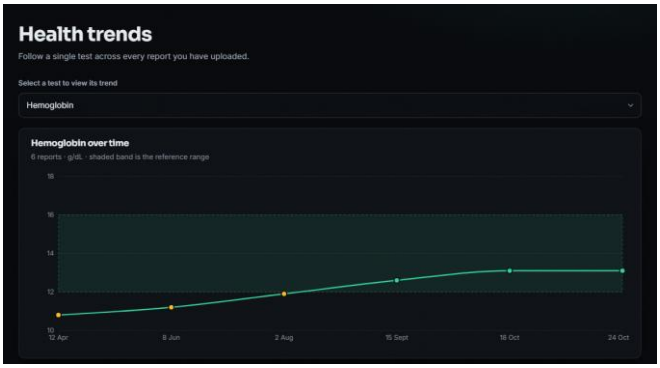


Fig. 5. Output screen: haemoglobin across six uploaded reports against its reference band.

Name matching. At 5, 10 and 20 % per-character noise over 55 names the shipped matcher is right in 96%, 89% and 73% of 30 trials per name, exact matching in 69%, 49% and 24%, and a plain difflib ratio at 0.60 in 99%, 98% and 95% (Fig. 6). The baseline is more noise-tolerant than the matcher that was shipped. Lowering the threshold from 0.55 to 0.30 lifts accuracy at 10 % noise from 89.0% to 96.5% but raises the share of 24 unseen names wrongly snapped to a dictionary entry from 17% to 54%; at 0.80 nothing is snapped wrongly but accuracy at 10 % noise falls to 70% (Fig. 7). Common abbreviations (Hb, TLC, LDL, HDL) are not matched, and only 53% of 19 synonyms are.

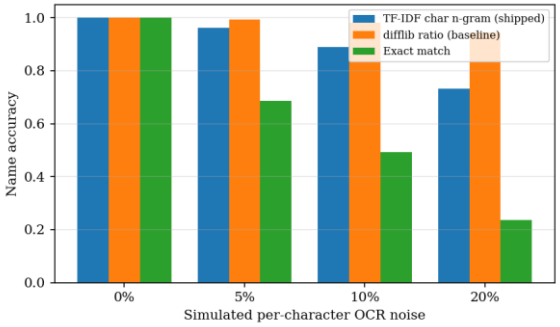


Fig. 6. Test-name accuracy under simulated character noise: shipped TF-IDF matcher, difflib baseline and exact match.

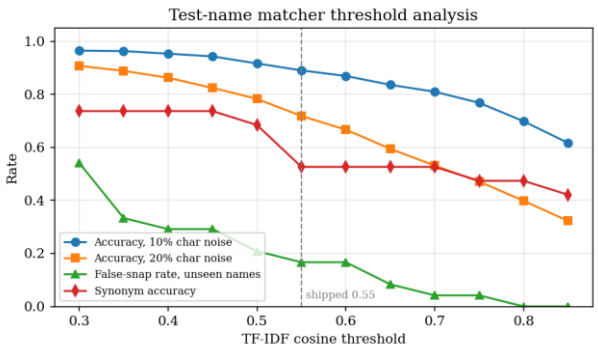


Fig. 7. Effect of the matching threshold on accuracy under noise, on unseen names wrongly snapped and on synonyms; the dashed line is the shipped 0.55.

Classification and explanations. Against an independently written rule, the classifier disagreed on none of 20,000 random cases, and inclusive bounds behave as specified. The explanation table covers 17 of 55 dictionary tests (30.9%); in the corpus only 202 of 565 abnormal dictionary rows (35.8%) receive test-specific text and the rest get the generic wording. Clinical correctness of the explanations was not evaluated.

Performance and testing. For a 14-row, one-page text PDF, the median time is 1.4 ms for text extraction, 7.2 ms for field extraction and 52.7 ms for the whole pipeline including the database (95th percentile 61.0 ms, 40 runs); field extraction grows linearly, from 8 ms at 14 rows to 113 ms at 200. Through HTTP on the loopback interface an upload takes 210 ms (95th percentile 414 ms), and the read endpoints 1-16 ms. 36 pytest tests and 31 smoke checks (24 API, 7 browser) pass. Scanned-image OCR accuracy and speed were not measured, because Tesseract is absent on the test machine; the <10 s target of the abstract is met for text-layer PDFs only (Fig. 8).

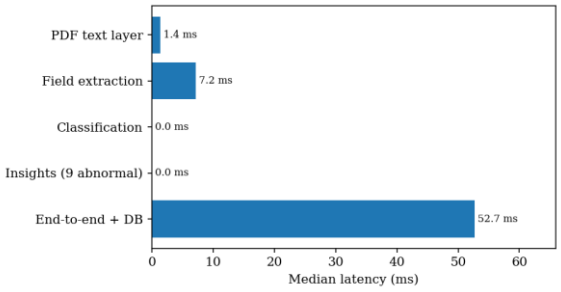


Fig. 8. Median latency of each pipeline stage for the 14-row report.

Worked example. The demo library holds five invented report PDFs (April to October 2026, 14 tests each) generated by the product's own seeding route, plus one uploaded file; all pass through the same pipeline. The dashboard then shows 6 reports and 82 extracted tests, of which 54 are normal, 28 abnormal and 2 critical. For the August report, 5 of 14 results are normal and 9 abnormal, each with an explanation (Fig. 4); haemoglobin rises from 10.8 to 13.1 g/dL over the six reports and is below its 12 g/dL limit in the first three (Fig. 5). Those values are demonstration data, not patient data.

Threats to validity. The corpus and the parser were written by the same team, so layouts the authors did not think of are absent; the generated names, units and ranges come from the same vocabulary the dictionary was built from, which favours the matcher; noise is simulated on a text layer, not produced by an optical scanner; the status label reuses the project's rule; and every figure comes from one run on one machine with a fixed seed, with no confidence intervals. The extraction numbers should therefore be read as a regression baseline for this code, not as an estimate of accuracy on real laboratories' reports. Security is demonstration-grade: one shared account, no per-user separation of reports and no transport encryption.

What the errors mean for a reader. Range and status accuracy are above 99 % among detected rows, so a value that is read is almost always flagged correctly. The costly errors are therefore silent ones: a missed row simply does not appear, so a user of a table-layout report sees nothing and is told that no recognizable rows were found, but one missing row in an otherwise complete report gives no warning. A date line read as a test is harmless to the flags (its range -4 to 2026 makes it normal) but clutters the list and the counts. A wrong name is worse, because the explanation that follows belongs to the wrong test, as when Sodium Bicarbonate receives the text for Sodium.

Comparison with the base paper. The two studies are not comparable. The base paper [1] scores 20 generated answers with GPT-4 and measures no extraction; this study scores extraction against generated labels and does not evaluate explanation quality. The base paper retrieves from a 20-book and 383-video knowledge base; here explanations come from a 17-entry table. What this work adds is a measured extraction stage, general parameters instead of 33 chosen for one syndrome, and stored history with trends.

V. CONCLUSION

Of the six objectives, flagging results against reference ranges (3) and tracking parameters over reports (5) were met and tested. Objective 1 was met in part: text-layer PDFs are read in milliseconds, but scanned-image OCR through Tesseract is unmeasured and PP-OCR is not used. Objective 2 was met in part: the parser is regular-expression based, not a trained CRF tagger. Objective 4 was met in part: explanations are curated rules for 17 tests, with no retrieval or citations, and the language-model switch is untested. Objective 6 was met in part: the dashboard exists, but accuracy was evaluated on generated reports, not manually labelled ones. The key finding is that field values are read reliably when each test sits on one line (F1 0.916 overall, range and status accuracy above 99 %), while units, table layouts, date lines and name matching are the weak points. Storage is a single-user SQLite file with a demo login, and the platform is educational, not a medical device.

Future work: parse table cells by position, recognise the unit and date patterns that failed, replace the name matcher by a better-tuned or trained one, widen the explanation base

with retrieval and citations, and measure OCR and the language-model switch on real, manually labelled reports.

REFERENCES

[1] A. Karhade, I. Verma, V. Gadaokar, and J. Mahajan, "RAG-based blood test report analysis to reverse metabolic syndrome," Commun. Appl. Nonlinear Anal., vol. 32, no. 10s, pp. 2771-2779, 2025, doi: 10.52783/cana.v32.5718.

[2] X. Ren, D. Dou, X. He, F. Cui, and J. Zhao, "A serialization method for digitizing the image-based medical laboratory report," Digital Health, 2025, doi: 10.1177/20552076251334431.

[3] N. Bakhsheshi and H. Beigy, "Analysis of blood report images using general-purpose vision-language models," arXiv:2509.06033, 2025.

[4] S. Xiang, S. Chen, K. Q. Lin, J. Yu, Y. Sun, P. Torr, and A. Atapour-Abarghouei, "Checkup2Action: A multimodal clinical check-up report dataset for patient-oriented action card generation," arXiv:2605.11533, 2026.

[5] Artifex Software, "PyMuPDF documentation." [Online]. Available: https://pymupdf.readthedocs.io

[6] G. Bradski, "The OpenCV library," Dr. Dobb's Journal of Software Tools, 2000.

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[8] S. Ramirez, "FastAPI framework documentation." [Online]. Available: https://fastapi.tiangolo.com

[9] Recharts contributors, "Recharts: A composable charting library built on React components." [Online]. Available: https://recharts.org

[10] R. Smith, "An overview of the Tesseract OCR engine," in Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), 2007, pp. 629-633.

[11] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459-9474.

[12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, 2019, pp. 3982-3992.

[13] M. Salvagno, F. S. Taccone, and A. G. Gerli, "Evaluating the feasibility of ChatGPT in healthcare: An analysis of multiple clinical and research scenarios," J. Med. Syst., vol. 47, no. 1, p. 33, 2023.

[14] V. X. Liu, C. A. Sinsky, S. Lin et al., "AI-generated draft replies integrated into health records and physicians' electronic communication," JAMA Netw. Open, vol. 7, no. 3, p. e2417615, 2024.

[15] H.-L. Hsu, C.-T. Dao, L. Wang et al., "MEDPLAN: A two-stage RAG-based system for personalized medical plan generation," arXiv:2503.17900, 2025.

[16] V.-T. Nguyen, H.-D. Pham, T.-H. To, C.-T. H. Do, T.-T.-T. Dong, V.-T. D. Le, and V.-P. Hoang, "Medalyze: Lightweight medical report summarization application using FLAN-T5-Large," arXiv:2505.17059, 2025.

[17] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, J. Zhou, Y. Bai, Z. Yu, Y. Yang, Q. Dang, and H. Wang, "PP-OCR: A practical ultra lightweight OCR system," arXiv:2009.09941, 2020.

[18] J. Lafferty, A. McCallum, and F. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in Proc. 18th Int. Conf. Machine Learning (ICML), 2001, pp. 282-289.

[19] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Inf. Process. Manag., vol. 24, no. 5, pp. 513-523, 1988.

[20] Anthropic, "Messages API reference." [Online]. Available: https://docs.anthropic.com